

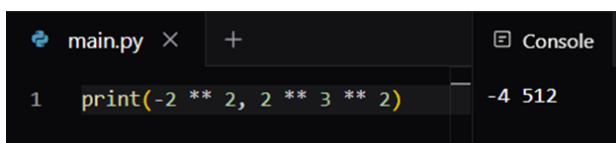
RELATÓRIO DA AULA 007

Quiz de abertura: o que cada linha imprime?

Slide 4:

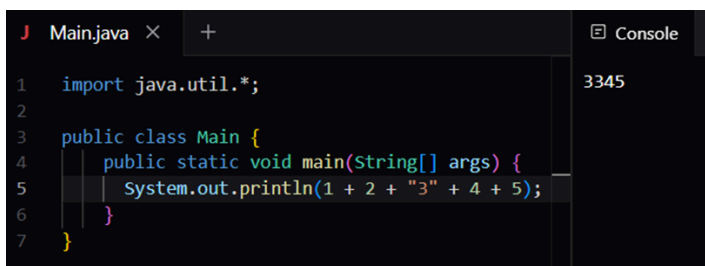
Este slide apresenta temos alguns códigos em linguagens diferentes e precisamos descobrir o que cada um vai imprimir.

Python:



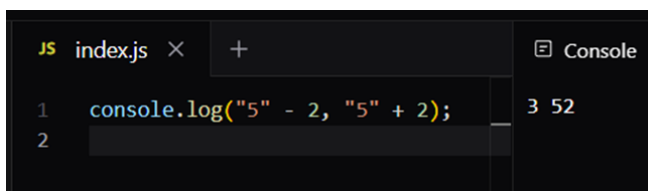
```
main.py x + Console
1 print(-2 ** 2, 2 ** 3 ** 2) -4 512
```

Java:



```
J Main.java x + Console
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         System.out.println(1 + 2 + "3" + 4 + 5);
6     }
7 }
```

Javascript:



```
JS index.js x + Console
1 console.log("5" - 2, "5" + 2); 3 52
2
```

C:



```
C Main.c x + Console
1 #include <stdio.h>
2 int main()
3 {
4     printf("%d\n", 3 > 2 > 1);
5 }
6
```

Python:



```
main.py x +
1 x = 0
2 print(x or "padrão", 3 > 2 > 1)
3
```

Console: padrão True

Javascript:



```
JS index.js x +
1 console.log(0 == "", "" == "0", 0 == "0");
2
```

Console: true false true

Expressões aritméticas:

Slide 5:

Neste slide o assunto principal é entender como uma linguagem decide a ordem das operações.

Precedência: define qual operação é feita primeiro. Ex.: $3 + 4 * 5 = 23$, pois a multiplicação vem antes da soma.

Associatividade: define a direção do cálculo quando os operadores têm a mesma prioridade, podendo ser da esquerda para a direita ou vice-versa.

Ordem dos operandos: define em que ordem as partes da expressão são avaliadas.

Efeitos colaterais: algumas operações podem alterar valores durante a execução da expressão.

Sobrecarga de operadores: algumas linguagens permitem definir novos comportamentos para operadores como +.

Mistura de tipos: cada linguagem possui suas próprias regras para trabalhar com tipos diferentes, como números e Strings.

Precedência e associatividade na prática:

Slide 6:

O slide mostra como **Python**, **JavaScript** e **Java** podem interpretar algumas expressões de maneiras diferentes, principalmente por causa das regras de precedência, associatividade e conversão de tipos.

Potência e sinal negativo: em Python, $-2 ** 2$ resulta em -4 , porque a potência é calculada primeiro: $-(2^2)$. No JavaScript, essa escrita gera erro de sintaxe, e no Java o operador ****** não existe.

Associatividade da potência: em $2 ** 3 ** 2$, o cálculo acontece da direita para a esquerda: $2 ** (3 ** 2)$, resultando em **512**.

Mistura de números e texto: em $1 + 2 + "3"$, Python gera um `TypeError`, pois não permite somar diretamente número com String. Já Java e JavaScript fazem a soma $1 + 2$ e depois a concatenação com "3", resultando em **"33"**.

Divisão com números negativos: os resultados podem variar. Por exemplo, Python com `//` resulta em -4, JavaScript em -3.5 e Java com inteiros em -3.

Operador de resto %: também existem diferenças. Para $-7 \% 2$, Python retorna 1, enquanto Java e JavaScript retornam -1.

Associatividade: operadores como $+$, $-$, $*$, $/$ e $\%$ geralmente são avaliados da esquerda para a direita. Já a potência $**$ é associada da direita para a esquerda em Python e JavaScript.

Obs: Quando a ordem de uma expressão não estiver clara, **usar parênteses ajuda a evitar erros e facilita a leitura do código**.

Ordem dos operandos e efeitos colaterais:

slide 7:

Ao compilar operações e métodos, cada linguagem possui uma abordagem diferente para realizá-la. O Java realiza a operação da esquerda para a direita, o C não possui ordem e cada compilador pode exibir um resultado diferente e o Go dita a ordem das chamadas de função, porém não a leitura das variáveis em relação a elas. Para esse problema, existem duas soluções, podendo proibir que efeitos colaterais ocorram ou fixar a ordem de compilação.

Para cada forma de compilar, cada uma apresenta vantagens e desvantagens, sendo a melhor aquela que se encaixa de forma mais adequada à aplicação.

Operadores sobrecarregados:

slide 8:

Em diferentes linguagens, a adição de funcionalidades ou significados pelo usuário para operadores são permitidos, com um operador podendo exercer mais que um método, com exceção de Java, onde o $+$ de Java serve para inteiros, reais e concatenação. Porém, exemplos de riscos de conflito como em C++, onde o $\&$ é E bit a bit e "endereço de". Segundo Sebasta, o uso múltiplo de um operador é aceitável desde que não prejudique a legibilidade nem a confiabilidade.

Conversões de tipos: alargamento, estreitamento e coerção:

Slide 9:

Alargamento: ocorre quando um tipo de variável é transformado em um maior, podendo ser aplicado em variáveis `int`, que podem ser transformadas em `double`. São relativamente seguras, porém podem perder precisão.

Estreitamento: ocorre a variáveis que são transformadas para outro tipo que não conseguem representar todos os seus valores originais, acontecendo perda de dados.

Coerção: possuem duas formas de acontecer, sendo explícita e implícita, com a implícita sendo a conversão de variável feita pela própria linguagem ou sistema e a explícita, também chamada de `Cast`, sendo a conversão feita pelo programador.

Em alguns casos, certas linguagens como `ML` e `F#` não permitem a conversão de tipos automática que se misturam entre si, enquanto `Java` converte `byte` e `short` através de coerção para `int` em operações aritméticas.

Coerção implícita: JavaScript e PHP:

Slide 10:

Primeiramente, o que é uma coerção implícita? Coerção é a conversão automática ou implícita de um valor de um tipo para outro, permitindo seu uso onde um tipo diferente é esperado. Muitas linguagens de programação suportam a conversão de um valor para outro de um tipo de dado diferente. Esse tipo de conversão pode ser feita implícita ou explicitamente. A conversão implícita, também chamada de coerção, é realizada automaticamente. A conversão explícita, também chamada de casting, é realizada por meio de instruções de código. Esse código trata uma variável de um tipo de dado como se pertencesse a um tipo de dado diferente. As linguagens que suportam conversão implícita definem as regras que serão aplicadas automaticamente quando valores primitivos compatíveis estiverem envolvidos. O código C abaixo ilustra a coerção implícita e explícita. Na linha 2, a constante ``int` `3`` é convertida automaticamente para ``double`` antes da atribuição (coerção implícita). Uma coerção explícita é realizada envolvendo o tipo de destino com parênteses, o que é feito na linha 3.

```
1 1 x duplo, y;  
2 2 x = 3; // coerção implícita (coerção)  
3 3 y = (double) 5; // coerção explícita (conversão de tipo)
```

Uma função é considerada polimórfica quando permite a coerção implícita ou explícita de parâmetros. Se o mesmo for válido para operandos, o operador correspondente é considerado polimórfico. Abaixo, um trecho de código C++ exemplifica essas expressões polimórficas.

```
1  #include <iostream>
2  void f(double x) { // função polimórfica
3      std::cout << x << std::endl;
4  }
5
6  int main() {
7      double a = 5 + 6.3; // operador polimórfico
8      std::cout << a << std::endl;
9
10     f(5);
11     f((duplo) 6);
12 }
```

O slide apresenta exemplos em JavaScript e em PHP

JavaScript

```
"5" - 2           // 3
"5" + 2           // "52"
"5" * "2"         // 10
true + 1          // 2
[] + {}           // "[object Object]"
null + 1          // 1
undefined + 1     // NaN
"b" + "a" + +"a" + "a" // "baNaNaN"

0 == ""          // true
"" == "0"        // false
0 == "0"         // true  (não é transitivo!)
0 === "0"        // false (=== não converte)
```

PHP · "type juggling"

```
var_dump("0e123" == "0e456"); // true !
var_dump("0e123" === "0e456"); // false
var_dump("1" == "01");       // true

md5("240610708") // "0e462097431906...."
md5("QNKCDZO")  // "0e830400451993...."
// == entre os dois hashes: true
```

Type juggling em PHP refere-se à conversão automática de valores entre tipos de dados realizada pelo PHP durante operações, como aritméticas ou de comparação. Isso também é conhecido como conversão implícita e ocorre automaticamente

durante a execução do programa. Regra prática: Em JavaScript e PHP, compare com `===` e `!==`, que não fazem coerção.

Expressões relacionais e booleanas:

Slide 11:

Primeiramente, o que são Expressões relacionais e booleanas? Uma expressão relacional é uma expressão utilizada para comparar dois valores. Essa comparação sempre produz como resultado um valor lógico, que pode ser verdadeiro ou falso. Já as expressões booleanas são expressões que trabalham com valores lógicos, normalmente representados por verdadeiro e falso. Elas podem ser formadas por uma única condição ou pela combinação de várias condições. Para combinar essas condições, utilizamos operadores lógicos, sendo os principais o AND, que significa “E”, o OR, que significa “OU”, e o NOT, que significa “NÃO”. No slide é apresentado o seguinte código:

C · booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c);    // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c));    // 1

if (-1) printf("verdadeiro\n");    // != 0
```

A expressão retorna 0 (primeiro código) porque o computador não avalia tudo de uma vez. Ele lê a expressão da esquerda para a direita, resolvendo uma comparação por vez. A segunda seria a maneira correta de fazer este código retornar verdadeiro.

Python · comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c)    # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0    0.0    ""    []    {}    None
# contam como VERDADEIRO:
# "0"    [0]    " "
```

Python possui um recurso chamado comparação encadeada. Quando você escreve `3 > 2 > 1`, o Python reescreve a expressão por debaixo dos panos usando o operador lógico `and`:

Ele entende como: `(3 > 2) and (2 > 1)`

Como `3 > 2` é True e `2 > 1` também é True, o resultado final é True.

Avaliação em curto-circuito:

Slide 12:

Primeiramente, o que é Avaliação em curto-circuito? A avaliação em curto-circuito, também conhecida como short-circuit evaluation, é uma técnica utilizada por muitas linguagens de programação para avaliar expressões lógicas. Seu principal objetivo é interromper a avaliação de uma expressão assim que o resultado final puder ser determinado, sem a necessidade de avaliar todas as partes da expressão. No slide possuímos esse exemplo em Java

Java · && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;           // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0    // false
s != null & s.length() > 0    // NullPointerException
```

O primeiro código consegue evitar graças ao uso de “&&” que trava a operação caso a primeira condição dê false; Com o segundo código a primeira condição, igual ao código anterior, trava a função e retorna false, já a segunda com apenas um “&”, é um AND lógico sem curto-circuito. Nesse caso, as duas expressões são avaliadas, com isso acontecendo e uma condição sendo dependente da outra, ocorre o erro de NullPointerException.

JavaScript · ?. e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade    // undefined, sem erro

const qtd = 0;
qtd || 10              // 10 (o 0 se perdeu!)
qtd ?? 10              // 0 (só troca null/undefined)
```

Em JavaScript temos uma ideia similar com os operados “?” e “??”

“?”. chamado de optional chaining, ou encadeamento opcional. Quando escrevemos: u.endereco?.cidade o JavaScript verifica primeiro se u.endereco possui um valor válido para acessar a propriedade. E “??”, chamado de nullish coalescing operator. não considera todo valor falsy como motivo para substituir o valor. Ele só substitui quando o valor é: null ou undefined; Portanto: 0 ?? 10, resulta em: 0, porque 0 não é null nem undefined.

Sentenças de atribuição:

Slide 13:

As sentenças de atribuição são fundamentais para armazenar e modificar dados em variáveis. Elas se apresentam nas formas simples, compostas, unárias (como incrementos) ou múltiplas. Em linguagens como C e JavaScript, a atribuição é tratada como uma expressão que retorna valor. Essa característica permite atalhos de escrita, mas pode induzir a erros ao confundir `=` com `==`. Em contrapartida, linguagens como Python e Go impõem restrições para evitar esse comportamento accidental. Python, por exemplo, reserva o uso do operador morsa (`:=`) para atribuições dentro de expressões. Essas escolhas de projeto refletem um equilíbrio entre flexibilidade de escrita e prevenção de falhas. Compreender tais particularidades é essencial para desenvolver códigos mais seguros, claros e eficientes.

Atribuição de modo misto:

Slide 14:

A atribuição de modo misto ocorre ao atribuir a uma variável um valor de tipo diferente. Quando o tipo do dado é ampliado (como um inteiro para double), linguagens como Java realizam a conversão automática (*widening*). Por outro lado, a conversão estreita, com potencial perda de dados, exige coerção explícita (*casting*) em linguagens mais rígidas, embora C permita esse processo implicitamente. Contudo, em Java, exceções como os operadores de atribuição composta (ex.: `+=`) executam conversões implícitas, o que demanda atenção. Outro risco comum reside na ordem de avaliação das operações: a divisão entre inteiros descarta a parte decimal antes mesmo da atribuição a uma variável real.

Estruturas de controle: quantas são necessárias?

Slide 15:

As estruturas de controle definem a ordem em que as instruções de um programa são executadas. Segundo o Teorema de Böhm e Jacopini, apenas três estruturas fundamentais são suficientes para representar qualquer algoritmo: sequência, seleção e iteração. A sequência executa comandos em ordem sucessiva, a seleção escolhe caminhos a partir de condições lógicas (*if/else*) e a iteração repete instruções enquanto uma condição for satisfeita (*while/for*). Essa fundamentação deu origem à programação estruturada e provou que o uso do comando *goto* é totalmente desnecessário. Embora essas três formas sejam suficientes, as linguagens modernas oferecem recursos adicionais como *switch*, *foreach* e *do-while*.

para melhorar a legibilidade do código. Além disso, as estruturas atuais priorizam ter apenas uma entrada para manter o fluxo claro. Por fim, a escolha entre as construções disponíveis deve sempre visar a clareza, a facilidade de leitura e a manutenção do programa.

Seleção de dois caminhos: o "else pendente":

Slide 16:

Em alguns programas que não possuem endentação correta, pode parecer que uma condicional else pertence ao if que está no mesmo nível, porém, em linguagens como Java, C e C++, um else sempre pertence ao if mais próximo que já não possui algum else, com a endentação ignorada. No caso de linguagens com chave obrigatória, como Perl, Go e Rust, o else pertence ao if de acordo com a chave.

Seleção múltipla: do switch de C ao match:

Slide 17:

A seleção múltipla permite que o programa escolha entre diferentes segmentos de código de acordo com o valor de uma expressão de controle. As linguagens possuem diferentes formas de definir e executar esses casos.

No C, o switch permite que a execução passe de um caso para o outro quando não há um break. Já no C#, essa passagem não acontece automaticamente. O Java 14+ também possui uma forma de switch que evita essa queda entre os casos e pode ser usada para retornar valores.

No Go, cada caso termina automaticamente, sendo necessário utilizar fallthrough para continuar no próximo. Já Rust e Python utilizam o match, que permite trabalhar com diferentes padrões e condições. No Rust, todos os casos possíveis precisam ser considerados.

Além disso, quando há muitos casos, o compilador pode utilizar estratégias mais eficientes para localizar o caso correspondente, em vez de verificar cada opção uma por uma.

Laços controlados por Contador:

Slide 18:

Quando um laço utiliza um contador (Geralmente laço for), este Contador pode funcionar de maneira lógica ou iterável.

Maneira lógica: O Contador do laço pode ser afetado por operações dentro do laço (Exemplo: Em um for em que i vai de 0 a 9, mas durante o laço soma 2 a i. No método lógico, i irá de 0 a 2, 4, 6, 8, 10, sendo executado apenas 5 vezes).

Maneira iterável: O Contador sempre somará 1 ao valor do laço anterior (Exemplo: Em um for em que i vai de 0 a 9, mas durante o laço soma 2 a i. No método iterável, i será executado 10 vezes, mesmo que seja modificado durante o laço).

Laços lógicos, saídas e iteradores:

Slide 19:

Java – break rotulado:

O exemplo percorre uma matriz com dois laços for e procura um elemento negativo. Quando o encontra, imprime sua posição e executa um break externo.

Um break sem rótulo encerraria apenas o laço interno. O laço externo poderia continuar examinando outras linhas da matriz. A escolha entre essas formas deve corresponder à intenção da busca: terminar toda a operação ou somente a repetição mais próxima.

JavaScript – for...in x for...of:

Para o vetor notas = [10, 20, 30], o for...in fornece as chaves "0", "1" e "2". Ao somá-las a um acumulador iniciado em 0, a operação passa a concatenar texto e produz "0012". O for...of fornece os valores 10, 20 e 30, permitindo obter a soma numérica 60.

O caso do Zune, ilustra um laço sem progresso. Quando dias vale 366 e o ano é bissexto, a condição dias > 365 continua verdadeira, mas o teste interno dias > 365 é falso. Nenhuma variável muda e a próxima repetição reencontra a mesma situação.

Desvio incondicional: o goto:

Slide 20:

Goto Fail:

O programa deveria verificar um hash e, depois, uma assinatura. Se alguma dessas etapas falhasse, o comando goto fail levaria a execução até o trecho que retorna o resultado. Porém, esse comando foi repetido logo após o primeiro if.

Como o if estava sem chaves, apenas o primeiro goto fail dependia da condição. O segurando era executado de qualquer forma. Com isso, o programa pulava a verificação da assinatura.

Se a verificação do hash retornasse zero, esse valor ficaria armazenado em erro e seria devolvido no final.

O slide apresenta break, continue e return como formas restritas de desvio. O break encerra a estrutura, o continue avança para a próxima interação e o return encerra a execução ou método. Essas operações expressam intenções específicas e facilitam a compreensão.

Comando protegidos de Dijkstra:

Slide 21:

Nos comandos protegidos de Dijkstra, cada ação tem uma condição, chamada guarda, que indica quando ela pode ser executada. Se mais de uma condição for verdadeira, qualquer uma das ações correspondentes pode ser escolhida, sem uma ordem fixa. Quando nenhuma condição é verdadeira, o resultado depende da estrutura: na seleção, ocorre um erro; na repetição, o laço termina. O select de Go segue uma ideia semelhante ao escolher entre operações de comunicação que estão prontas para acontecer.

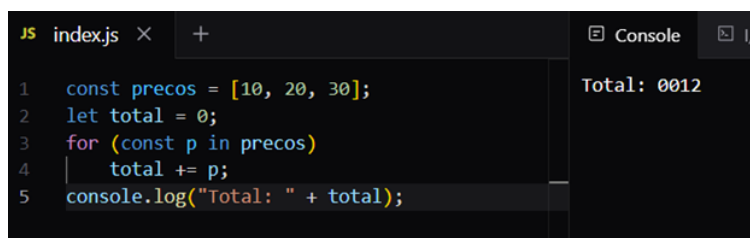
Exercício em duplas: encontre o bug:

Slide 22:

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

JavaScript:

a)



The screenshot shows a code editor with a file named 'index.js'. The code is as follows:

```
1 const precos = [10, 20, 30];
2 let total = 0;
3 for (const p in precos)
4   total += p;
5 console.log("Total: " + total);
```

The console output on the right shows 'Total: 0012'.

b) O problema está no uso do **for...in**, que percorre as propriedades/índices do array, junto com a conversão de tipos do JavaScript.

c) Usar **for...of**, que percorre os valores:

```
JS index.js x + Console
1 const precos = [10, 20, 30];
2 let total = 0;
3
4 for (const p of precos)
5   total += p;
6
7 console.log("Total: " + total);
```

Total: 60

Python:

a)

```
main.py x + Console
1 def desconto(preco, d=None):
2     d = d or 10 # padrão: 10%
3     return preco * (100 - d) / 100
4
5 print(desconto(200, 0))
6
```

180.0

b) Em Python, o número 0 é considerado um valor **falso**. Então: `d = d or 10`, quando `d = 0`, acaba escolhendo 10.

c) Verificar especificamente se `d` é `None`:

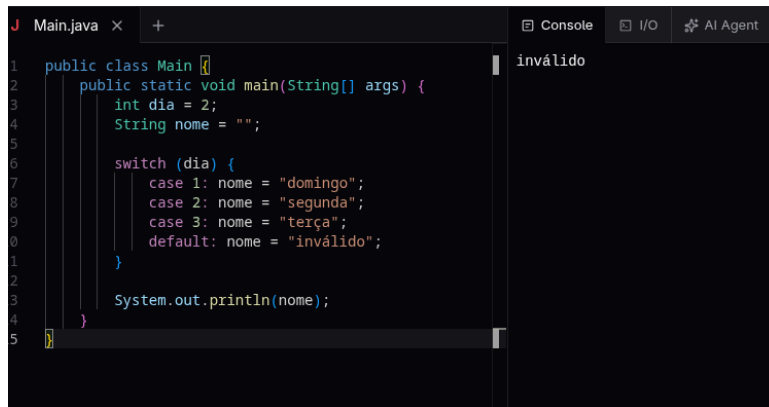
```
main.py x + Console
1 def desconto(preco, d=None):
2     if d is None:
3         d = 10
4     return preco * (100 - d) / 100
5
6 print(desconto(200, 0))
7
```

200.0

slide 23:

Java:

a)

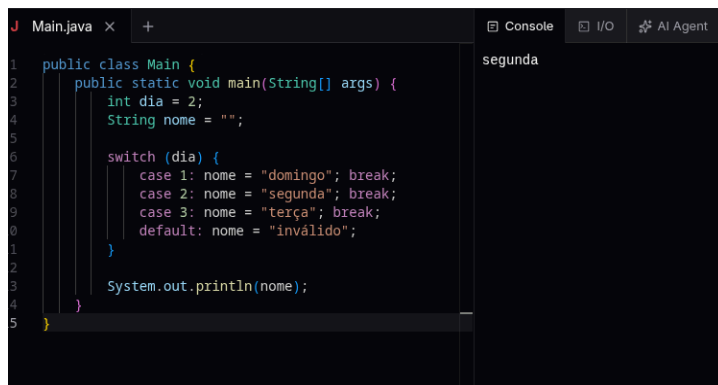


```
1 public class Main {
2     public static void main(String[] args) {
3         int dia = 2;
4         String nome = "";
5
6         switch (dia) {
7             case 1: nome = "domingo";
8             case 2: nome = "segunda";
9             case 3: nome = "terça";
10            default: nome = "inválido";
11        }
12
13        System.out.println(nome);
14    }
15 }
```

inválido

b) faltam os break. O programa entra no case 2 e continua executando os próximos casos, incluindo default, que troca o nome para “inválido”.

c) usar break nos case.



```
1 public class Main {
2     public static void main(String[] args) {
3         int dia = 2;
4         String nome = "";
5
6         switch (dia) {
7             case 1: nome = "domingo"; break;
8             case 2: nome = "segunda"; break;
9             case 3: nome = "terça"; break;
10            default: nome = "inválido";
11        }
12
13        System.out.println(nome);
14    }
15 }
```

segunda

C:

a)



```
1 #include <stdio.h>
2 int main(){
3     int saldo = 100, saque = 50;
4     if (saque > saldo);
5     printf("saldo insuficiente\n");
6     printf("fim\n");
7 }
```

saldo insuficiente
fim

b) Logo após o if tem um ; fazendo com que o seu corpo seja uma instrução vazia. Assim a mensagem “saldo insuficiente” aparece independentemente da condição.

c)

```
C Main.c x + Console I/O AI Agent
1 #include <stdio.h>
2 int main(){
3     int saldo = 100, saque = 50;
4
5     if (saque > saldo) {
6         printf("saldo insuficiente\n");
7     }
8     printf("fim\n");
9 }
```

slide 24:

Go:

a)

```
package main
import "fmt"

func main() {
    x := 10
    if x > 5 {
        x := x * 2
        fmt.Println("dentro:", x)
    }
    fmt.Println("fora:", x)
}
```

dentro: 20
fora: 10

b) Sombreamento de Variáveis (*Variable Shadowing*): Dentro do bloco if, foi utilizado o operador de declaração curta `:=` (`x := x*2`). Isto declara uma nova variável local `x` limitada ao escopo do if, em vez de atualizar a variável `x` do escopo externo (que permanece com o valor 10).

c) Para solucionar é só utilizar o operador `=` no lugar do `:=`

```
1 package main
2 import "fmt"
3
4 func main() {
5     x := 10
6     if x > 5 {
7         x = x * 2
8         fmt.Println("dentro:", x)
9     }
10    fmt.Println("fora:", x)
11
12 }
```

dentro: 20
fora: 20

Java:

a)

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        int acertos = 7, total = 10;
        double taxa = acertos / total * 100;
        System.out.println(taxa + "%");
    }
}
```

0.0%

b) No Java toda divisão de inteiro vai dar um valor inteiro, e como 7/10 seria 0.7, que seria 0, que então ficaria 0 * 100, no final o 0 é convertido de int para double.

c) Para resolver é só fazer um casting no código, que então corrigiria o problema

```
1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          int acertos = 7, total = 10;
6          double taxa = (double)acertos / total * 100;
7          System.out.println(taxa + "%");
8      }
9  }
```

70.0%

Síntese – Armadilhas e como as linguagens ajudam:

Slide 25:

O slide mostra alguns erros comuns na programação e como diferentes linguagens tentam evitar ou diminuir esses problemas.

- **Precedência:** a ordem dos operadores pode mudar o resultado de uma expressão. Usar parênteses ajuda a deixar a intenção mais clara.
- **Operandos:** funções com efeitos colaterais podem causar resultados inesperados dependendo da ordem de execução. Algumas linguagens, como Java, definem essa ordem.
- **Conversões:** linguagens como JavaScript podem fazer conversões automáticas de tipos, o que pode causar confusão. Go e Rust exigem conversões mais explícitas.
- **Booleanos:** valores como 0, "" e listas vazias podem ser considerados falsos em algumas linguagens. Outras, como Java, Go e Rust, são mais rígidas e exigem booleanos nas condições.

- **Atribuição:** escrever `if (x = y)` pode ser confundido com uma comparação. Algumas linguagens evitam isso porque a atribuição não pode ser usada dessa forma.
- **Seleção:** estruturas como `switch` podem gerar erros quando esquecemos o `break`. Outras estruturas, como `match`, possuem regras mais seguras.
- **Laços:** no JavaScript, usar `for...in` em arrays pode percorrer os índices em vez dos valores. Para valores, normalmente usamos `for...of`.
- **Desvio:** o uso de `goto` sem controle pode deixar o código confuso. Estruturas como `break`, `continue` e `return` deixam o fluxo mais organizado.

Atividades complementares (autopesquisa):

Slide 26:

C – switch

```
switch (dia) {  
    case 1:  
        printf("Segunda");  
        break;  
    case 2:  
        printf("Terça");  
        break;  
    case 3:  
        printf("Quarta");  
        break;  
    default:  
        printf("Outro dia");  
}
```

Java – switch

```
switch (dia) {  
    case 1 -> System.out.println("Segunda");  
    case 2 -> System.out.println("Terça");  
    case 3 -> System.out.println("Quarta");  
    default -> System.out.println("Outro dia");  
}
```

Go – switch

```

switch dia {
✓ case 1:
|   fmt.Println("Segunda")
✓ case 2:
|   fmt.Println("Terça")
✓ case 3:
|   fmt.Println("Quarta")
✓ default:
|   fmt.Println("Outro dia")
}

```

Rust – match

```

1 fn main() {
2     match dia {
3         1 => println!("Segunda"),
4         2 => println!("Terça"),
5         3 => println!("Quarta"),
6         _ => println!("Outro dia"),
7     }
8 }

```

Comparação:

Linguagem	Cai entre casos?	Exaustividade	Legibilidade
C	Sim, se não usar break	Não exige todos os casos	Simples, mas exige atenção ao break
Java ->	Não	Pode exigir cobertura quando usado como expressão	Clara e sem necessidade de break
Go	Não automaticamente	Não exige todos os casos	Simples e direta
Rust match	Não	Exige que todas as possibilidades sejam cobertas	Clara e segura, principalmente para muitos casos